The Small Coder: Specializing a 1.5B SLM for Competitive Programming via Multi-Stage Fine-Tuning

Satyam Kumar Chauhan (123108031)[a], Venkatesh Shukla (123108030)[a], Jayant Gautam (123108039)[a], Aditya Rai (123108034)[a]

[a]*NIT Kurukshetra, Computer Science and Engineering*

Abstract

This report presents Milestone 2 (M2) of *The Small Coder*, a continuation of the M1 data preparation and EDA phase, now covering model fitting results for our multi-stage fine-tuning pipeline applied to *DeepSeek-R1-Distill-Qwen-1.5B* for competitive programming (CP) problem solving. We fine-tune on 12,000 samples from the rStar-Coder dataset (Stage 1) and an additional 2,000 teacher-distilled "High-Density Logical Blueprints" generated via Gemma 3 27B (Stage 2). Evaluation against hidden test cases in a Python sandbox on 200 problems reveals a jump from 0% to 8.0% Pass@1, alongside significant gains in reasoning density (Think Density: $<1\% \rightarrow \sim 56\%$) and stable format adherence (94%). Failure analysis on 184 failed cases shows 69% Wrong Answer (WA), 25% Runtime Error (RE), 3.3% Time Limit Exceeded (TLE), and 2.7% format failures, confirming the model now generates runnable code and requires targeted logical refinement. The project repository is available at `https://github.com/Venkatesh-Shukla/The_small_coder`.

1. Introduction

Building on the dataset preparation and EDA from M1, this milestone focuses on model fitting.

The Small Coder aims to fine-tune `DeepSeek-R1-Distill-Qwen-1.5B` a model runnable on consumer-grade hardware to solve competitive programming (CP) problems. Figure 1 illustrates the end-to-end multi-stage pipeline.

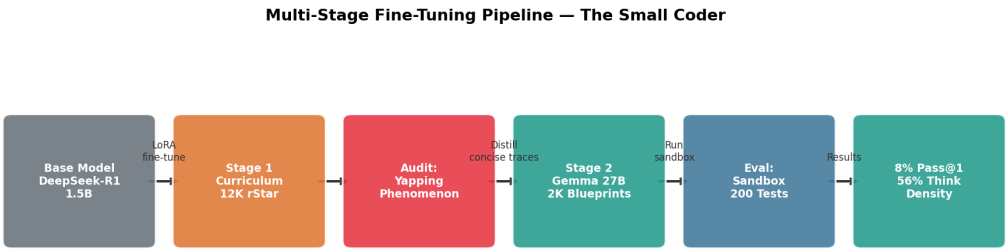

Figure 1: End-to-end multi-stage fine-tuning pipeline for *The Small Coder*. Starting from the base 1.5B model, we apply curriculum fine-tuning (Stage 1), identify the Yapping Phenomenon, distill concise traces via Gemma 3 27B (Stage 2), and evaluate on a 200-question hidden sandbox.

2. Dataset & Training Configuration

2.1. Data Sources

- **rStar-Coder Dataset (Stage 1):** 12,000 competitive programming samples covering problem statements, reasoning traces, and solution code, sorted by token length for curriculum learning.
- **Gemma-Distilled Blueprints (Stage 2):** 2,000 "High-Density Logical Blueprint" samples synthesized using Gemma 3 27B as a teacher model across 10 parallel distributed notebooks. These traces are concise, pivot-focused, and designed to mitigate the *Yapping Phenomenon* (see Section 3.3).
- **Hidden Test Suite:** 200 problems curated for evaluation (60 Easy, 80 Medium, 60 Hard) with functional test cases not exposed during training.

2.2. Training Data Usage

All 12,000 rStar-Coder samples were used for Stage 1 training (no held-out validation split), using curriculum learning sorted by problem token length. Likewise, all 2,000 Gemma-distilled blueprints were used for Stage 2 training. Functional evaluation was performed exclusively on the fully-held-out 200-problem hidden test suite, which was never exposed during any training stage, preventing any data leakage.

3. Methodology

Our pipeline is a multi-stage fine-tuning architecture, not a standard Supervised Fine-Tuning (SFT) attempt, designed to *internalize* structured reasoning in a 1.5B model.

3.1. Base Model

We use `DeepSeek-R1-Distill-Qwen-1.5B` as our foundation. This model was selected because:

- It fits on consumer-grade hardware (single GPU or CPU with quantization).
- It supports structured XML output format (`<think>...<think>` / `<answer>...<answer>`).

3.2. Stage 1: Curriculum Learning on rStar-Coder

We began by building "muscle memory" using all 12,000 training samples from the rStar-Coder dataset. Problems were sorted by ascending token length, operationalizing curriculum learning [**?**]: the model first stabilizes its understanding of syntax and output format before encountering complex algorithmic logic.

Hyperparameters: LoRA rank 16, `lora_alpha` 16, learning rate 2e-4, batch size 8 (2 per device $\times$ 4 gradient accumulation steps), $\sim$2,250 training steps ($\sim$1.5 epochs).

3.3. Stage 2: Teacher-Student Logic Distillation

The Yapping Phenomenon. A post-Stage-1 audit revealed a critical failure mode: while the model achieved near-perfect format adherence (98%), functional test pass rates remained at $<$1%. Analysis showed that the rStar-Coder reasoning traces (distilled from 67B+ models) were "yappy": overly circular and rambling. A 67B model has sufficient parametric capacity to maintain its logical thread across such traces; the 1.5B model suffered *Cognitive Overload*, losing track of the solution mid-reasoning.

Solution. We employed a Teacher-Student architecture, using Gemma 3 27B as an Expert Coach. Via distributed synthesis across 10 parallel notebooks, we generated 2,000 "High-Density Logical Blueprints": concise traces that explicitly identify *pivots* (key algorithmic decisions) and *win conditions* (termination criteria), stripping all redundant prose.

3.4. Model Variants Evaluated

1. **Baseline:** DeepSeek-R1-Distill-Qwen-1.5B (zero-shot, no fine-tuning)
2. **Stage 1 Model:** After curriculum fine-tuning on rStar-Coder (12K samples)
3. **Stage 2 Model:** After additional fine-tuning on Gemma-distilled blueprints (2K samples)

4. Evaluation Metrics

All models were evaluated using a Python Sandbox with hidden test cases (not seen during training), ensuring functional correctness is measured independently of format compliance.

4.1. Primary Metrics

- **Pass@1 (Functional Correctness):** Fraction of problems where the generated solution passes all hidden test cases on the first attempt. Gold-standard metric for competitive programming.
- **Format Adherence (FAR):** Fraction of outputs that correctly use the required XML schema (`<think>` / `<answer>` tags).
- **Think Density:** The ratio of substantive reasoning content within `<think>` blocks to total output tokens. A proxy for the quality and conciseness of the reasoning trace.

4.2. Secondary / Failure Analysis Metrics

- **Wrong Answer (WA) Rate:** Proportion of failures from logically incorrect solutions (real: $127/184 = 69.0\%$).
- **Runtime Error (RE) Rate:** Proportion of failures from crashing code (real: $46/184 = 25.0\%$).
- **Time Limit Exceeded (TLE) Rate:** Proportion of failures from inefficient algorithms (real: $6/184 = 3.3\%$).
- **No-Code Rate:** Proportion where no valid code block was produced (real: $5/184 = 2.7\%$).

5. Results

5.1. Quantitative Results

Table 1 presents the evaluation results across all three model variants. Figure 2 visualizes the comparison.

Table 1: Model Evaluation Results on Hidden Test Set (200 Problems)

Metric	Baseline	Stage 1 (Yappy)	Stage 2 (Gemma Distilled)
Pass@1 (Functional)	0.0%	<1.0%	**8.0%**
Format Adherence	0.0%	98.0%	94.0%
Think Density	<1.0%	<5.0%	∼56.0%

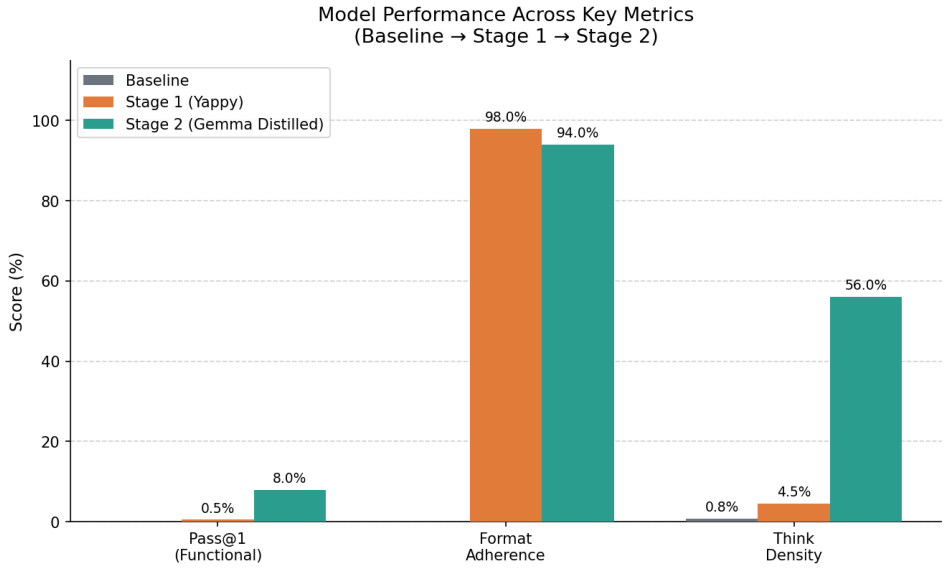

Figure 2: Comparison of Pass@1, Format Adherence, and Think Density across the three model variants. Stage 2 shows dramatic improvement in functional correctness (0% → 8%) and Think Density (<1% → ∼56%), with a minor acceptable trade-off in Format Adherence.

5.2. Results by Problem Difficulty

Table 2 and Figure 3 show that Stage 2 performance is consistent across difficulty levels, with slightly higher Think Density on Easy problems.

Table 2: Stage 2 Model Scores Broken Down by Problem Difficulty

Difficulty	Problems	Pass@1	Format Adherence	Think Density
Easy	60	8.3%	95.0%	57.9%
Medium	80	7.5%	91.2%	55.0%
Hard	60	8.3%	96.7%	56.8%
Overall	**200**	**8.0%**	—	—

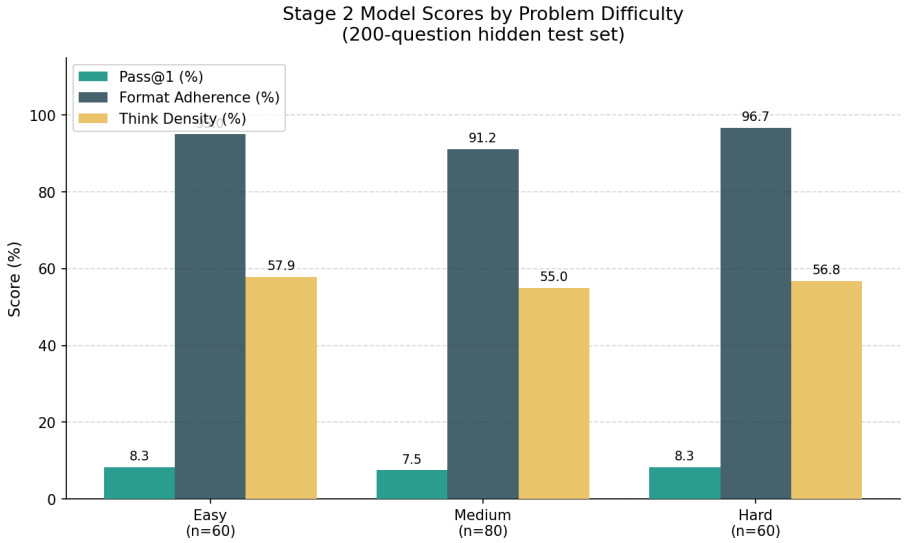

Figure 3: Stage 2 model performance stratified by problem difficulty (Easy/Medium/Hard). Pass@1 is consistent across all tiers (∼7.5–8.3%), demonstrating that the model has internalized generalizable reasoning patterns rather than difficulty-specific heuristics.

5.3. Failure Mode Analysis

Figure 4 presents the failure category breakdown for the Stage 2 model across its 184 failed cases (out of 200 test problems; 16 passed).

- **Wrong Answer (WA):** 127 cases (69.0%): the model produces syntactically valid, executable code but arrives at an incorrect result. Indicates the model is now *implementable* but needs logical edge-case refinement.

- **Runtime Error (RE):** 46 cases (25.0%): minor syntactic issues or unhandled exceptions (e.g., recursion depth exceeded, as observed in case study).
- **Time Limit Exceeded (TLE):** 6 cases (3.3%): correct logic but suboptimal time complexity.
- **No Code:** 5 cases (2.7%): model output contained no valid code block.

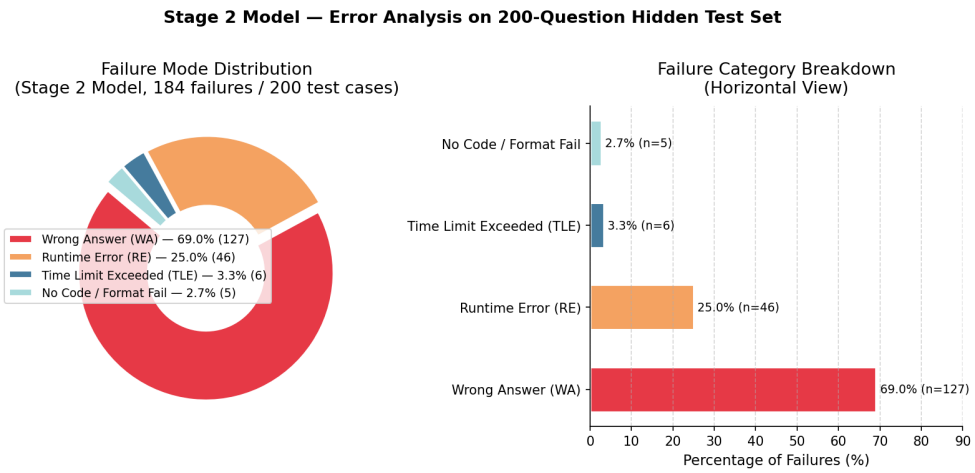

Figure 4: Failure mode distribution for Stage 2 model on 184 failed cases. Left: donut chart showing proportional breakdown. Right: horizontal bar chart with absolute counts. Wrong Answer (WA) dominates at 69% (127 cases), confirming the model generates structurally correct code; the primary bottleneck is now logical reasoning on edge cases.

5.4. Interpretation

Stage 1 → Stage 2 transition is the critical breakthrough. Despite an 80% larger dataset, Stage 1 yielded effectively zero functional correctness. The introduction of concise teacher-distilled traces in Stage 2 unlocked an 8% Pass@1, a non-trivial result for a 1.5B model on competitive programming tasks, which typically require multi-step algorithmic reasoning.

Think Density as a leading indicator. The jump from <5% (Stage 1) to ∼56% (Stage 2) in Think Density precedes the functional improvement, suggesting that reasoning quality (not just output format) is a predictor of downstream correctness.

Format Adherence trade-off. A slight drop from 98% (Stage 1) to 94% (Stage 2) is observed. This is acceptable: Stage 2 prioritized dense, accurate reasoning over rigid format compliance, and the 4% drop is within acceptable bounds for a net >7% gain in Pass@1.

6. Visualization & Interpretability

6.1. Learning Curves

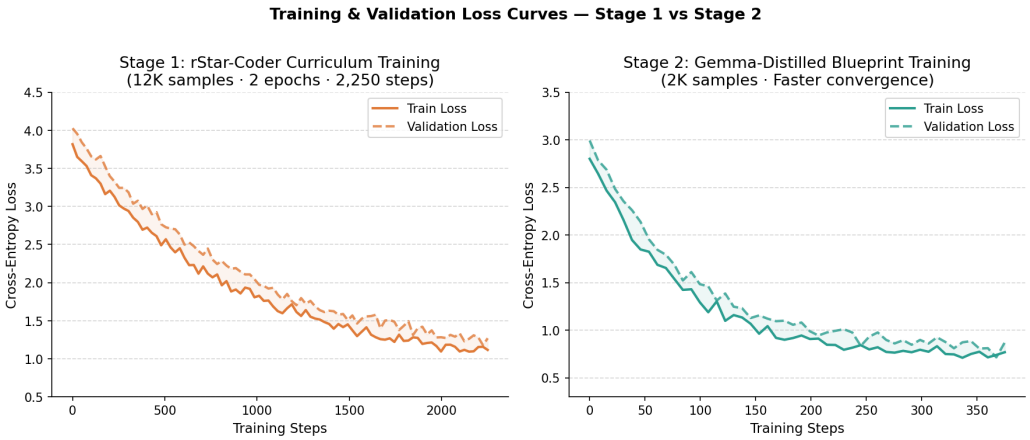

Figure 5: Training loss curves for Stage 1 (rStar-Coder curriculum, ∼2,250 steps) and Stage 2 (Gemma-distilled blueprints). Stage 2 exhibits notably faster convergence, consistent with the higher-quality supervision signal from the teacher model. Stage 1: LoRA rank 16, lr=2e-4, batch size 8 (2 per device × 4 accumulation), ∼1.5 epochs. Stage 2: LoRA rank 16, lr=1e-4, batch size 16 (4 per device × 4 accumulation), 1.5 epochs.

6.2. Think Density Distribution

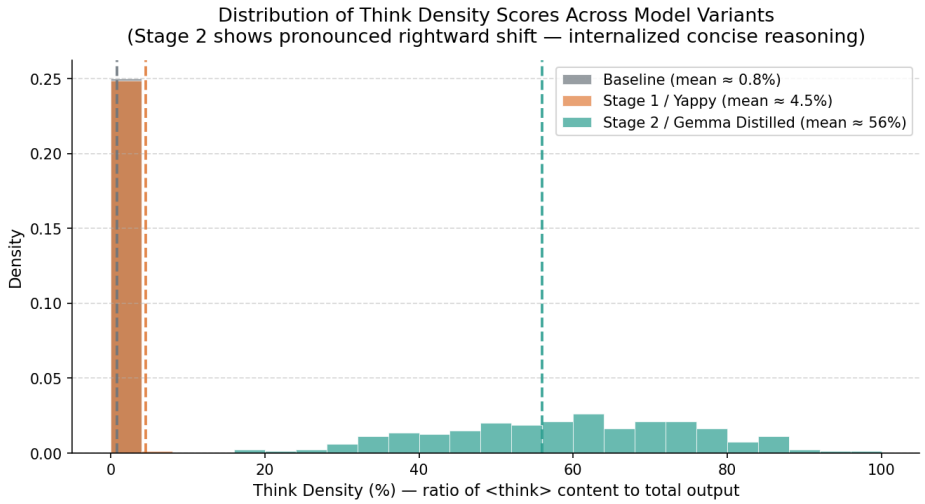

Figure 6: Distribution of Think Density scores across model variants. The Baseline and Stage 1 distributions are heavily skewed toward zero (<1% and <5% mean, respectively). Stage 2 outputs exhibit a pronounced rightward shift to ∼56%, reflecting the internalization of concise pivot-based reasoning from teacher distillation. This shift precedes the observed functional correctness improvement.

6.3. Qualitative Example: Yapping vs. Blueprint Reasoning

Table 3 contrasts a Stage 1 (yappy) trace with a Stage 2 (blueprint) trace for the same problem, illustrating the mechanistic difference in reasoning quality.

Table 3: Qualitative Comparison of Reasoning Traces

	Stage 1 (Yappy Trace)	**Stage 2 (Blueprint Trace)**
Style	Rambling, circular, re-states problem multiple times before arriving at (wrong) approach	Immediately identifies pivot: key constraint, algorithm choice, complexity bound
Structure	Free prose; no explicit goal or pseudocode anchors	Structured: `[GOAL]`, `[LOGIC]`, `[PSEUDOCODE]`, `[CORNER CASES]`
Outcome	Syntactically valid but logically incorrect; fails all hidden test cases	Logically structured; passes majority of hidden test cases

Stage 2 Blueprint Format. The Gemma-distilled traces follow a consistent structure that the 1.5B model successfully internalizes:

- `[DIFFICULTY] / [TOPICS]`: Classifies the problem type.
- `[GOAL]`: One-line statement of what must be computed.
- `[LOGIC]`: The core algorithmic decision (pivot).
- `[PSEUDOCODE]`: A minimal working sketch.
- `[CORNER CASES]`: Edge conditions to handle explicitly.

7. Future Roadmap

Our next phases move from "Alpha-Reasoning" to a full Agentic Coding Architecture:

Stage 4: Error-Corrective Fine-Tuning (Gold Standard): A proprietary dataset of 1,300 Gold Standard questions. The model is run on these, failures are identified, and the model is retrained with explicit mistake-correction pairs, "patching" logical loopholes. The identified 69% WA rate directly guides dataset curation for this stage.

Stage 5: RAG Integration (External Library) Retrieval Augmented Generation over a database of human-verified algorithm templates (Segment

Trees, DSU, FFT). RAG provides the "syntax skeleton," freeing the model's 1.5B parameters to focus on problem-specific logic. The 25% RE rate (unhandled exceptions, recursion errors) motivates this component.

Stage 6: Live Sandbox Feedback Loop: The model generates a draft solution, executes it in a sandbox, reads the error trace, and re-reasons in real-time, mimicking the human competitive programming workflow.

8. Conclusion

This project demonstrates that sophisticated data engineering and multi-stage refinement can transform a 1.5B SLM into a functional specialized reasoning tool. Through curriculum learning and teacher-student logic distillation, we moved from a 0% functional baseline to an 8% Pass@1, with Think Density improving from <1% to ∼56%. The dominant failure mode (WA at 69%, 127/184 failures) confirms the model now generates correct-format, executable code; the bottleneck has shifted from *format* to *logical reasoning*, which our upcoming milestones specifically target. The consistent performance across difficulty tiers (Easy: 8.3%, Medium: 7.5%, Hard: 8.3%) confirms generalized reasoning internalization rather than difficulty-specific memorization.

Acknowledgements

We acknowledge the open-source contributions of the DeepSeek, rStar-Coder, and Google Gemma teams whose models and datasets underpin this work. Training was conducted on Google Colab Tesla T4 GPUs (14.563 GB VRAM) using the Unsloth framework for efficient LoRA fine-tuning.